hashlock.

Security Audit

W30 (DeCommerce)

July 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

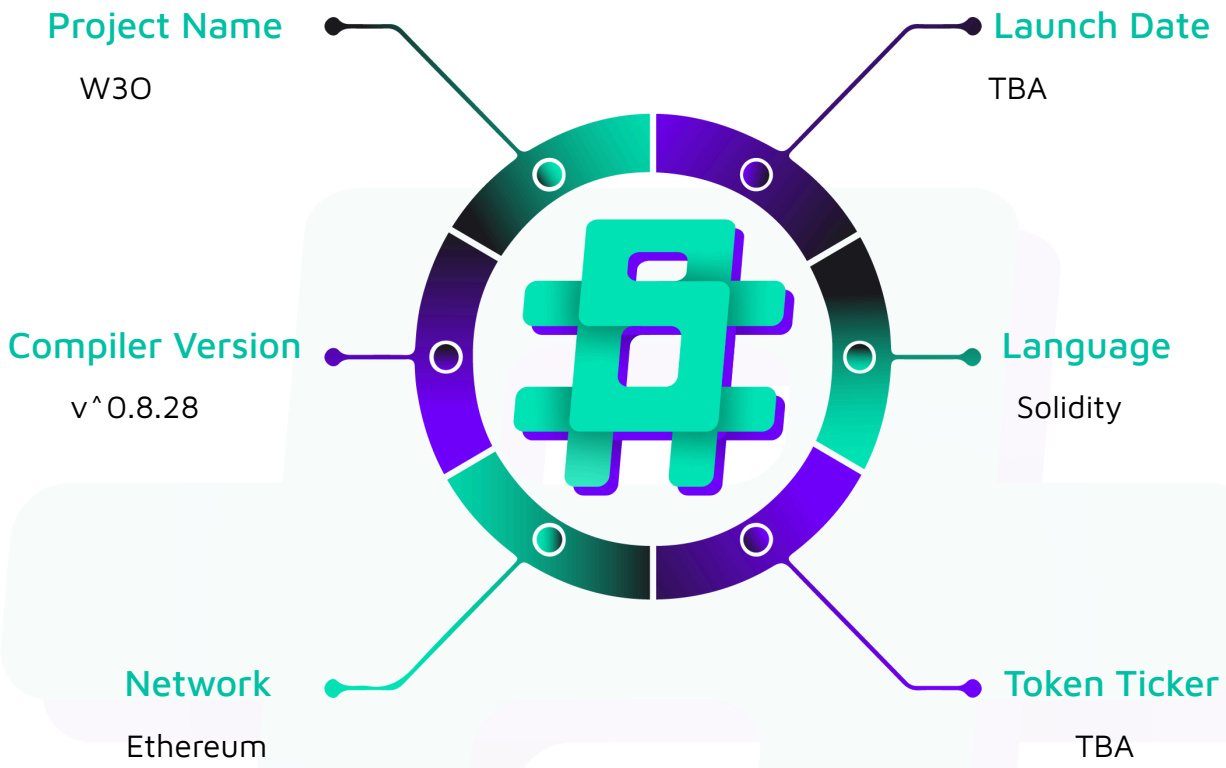
The W3O team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

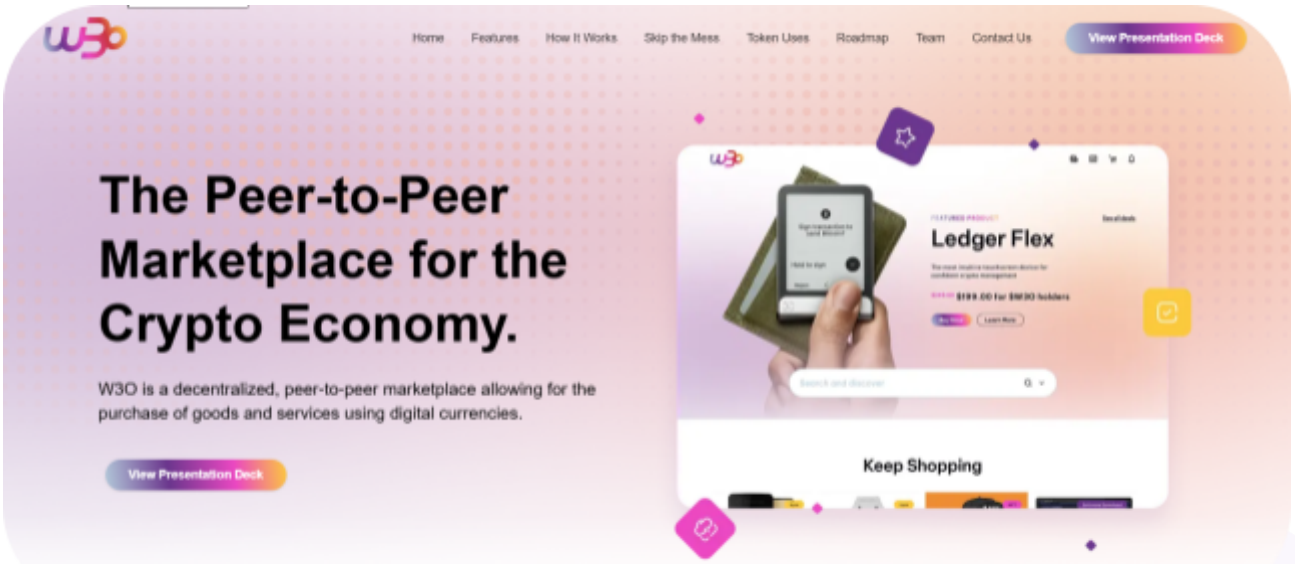
W3O is a Web3 decentralized marketplace that enables users to buy and sell physical goods, digital assets, and services directly using cryptocurrencies. The platform leverages smart contracts to provide non-custodial escrow, automated settlements, on-chain reputation management, and decentralized dispute resolution, aiming to reduce fees, eliminate intermediaries, and facilitate secure peer-to-peer global commerce. Its ecosystem is also powered by the $W3O token, which is used for payments, governance, staking, and marketplace incentives.

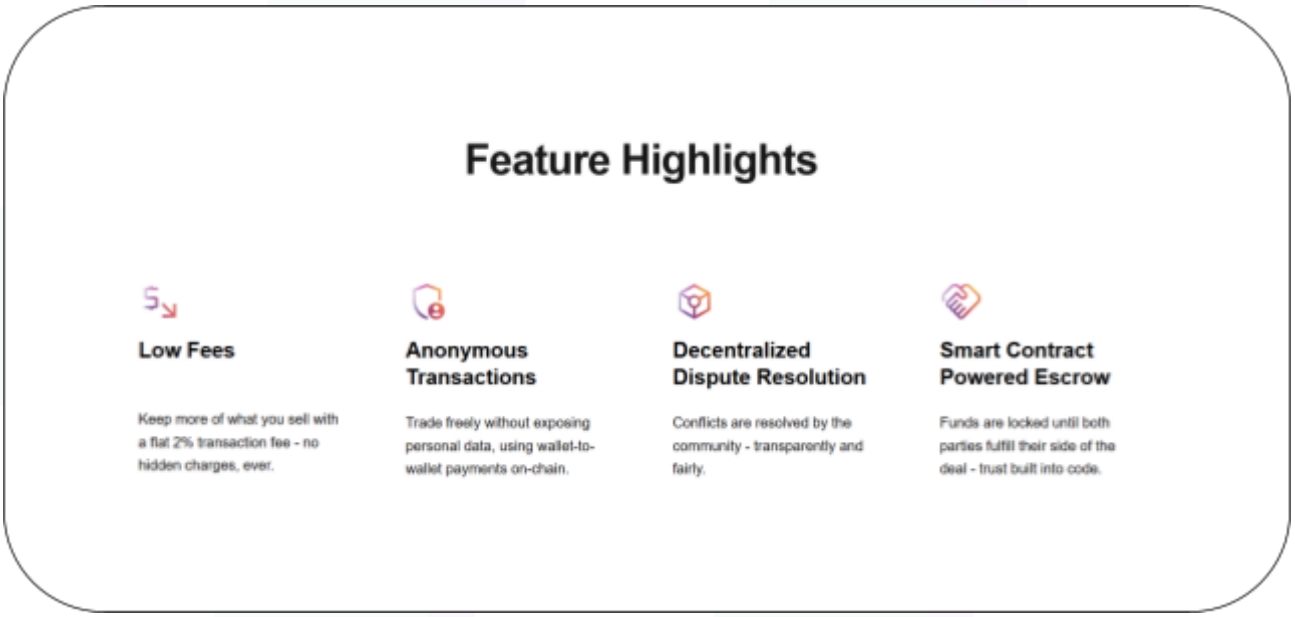
Project Name: W3O
Project Type: DeCommerce
Compiler Version: ^0.8.28
Website: https://w3o.store/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the W3O project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**W3O Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**July, 2026**
Contract 1	W3OPresale.sol
Audited GitHub Commit Hash	75940cb0ebc155339984e811596074ef0c6f791a
Fix Review GitHub Commit Hash	3a24484d9a4d2aa185e3e62ca629b1de0b3da654

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

5 Gas Optimisations

5 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

W3OPresale.sol
• Allows the `owner` (setter / governance, via `Ownable2Step`) to:
  ○ Halt all token purchases through `pause`, and resume them through `unpause`.
  ○ Rotate the off-chain signing authority that authorises every purchase, through `updateAdminSigner`.
  ○ Change the address that receives all sale proceeds, through `updateTreasury`.
  ○ Raise or lower the total presale allocation through `updateHardCap`, which refuses any new cap below `tokensSold`.
  ○ Transfer ownership under the two-step propose / accept handshake inherited from `Ownable2Step`.
• Allows the `adminSigner` (off-chain, via EIP-712 signature) to:
  ○ Authorise an individual purchase by signing a `Buyw3oMessage` or `Buyw3oWithEthMessage` payload naming the buyer, the payment token, the amount paid, the amount of `w3o` credited, a timestamp and a salt.
  ○ Set the exchange rate and any tiered purchase bonus for that specific
	Contract achieves this functionality.

purchase, since both `_amountPaid` and `_amountReceived` are fixed inside the signed payload.

- Allows users to:
  - Purchase `w3o` with an ERC-20 payment token through `buyw3o`, which validates the signature, forwards `_amountPaid` to `treasury`, and credits `_amountReceived` to `userTotalw3o`.
  - Purchase `w3o` with native ETH through `buyw3oWithEth`, which requires `msg.value` to equal the signed `_amountPaid` and forwards it to `treasury`.
  - Consume each signature exactly once — `usedSignatures` records the hash of every redeemed signature and rejects any resubmission.
  - Purchase only within the one-hour validity window, enforced as `block.timestamp <= _timeSigned + 1 hours`.
  - Purchase only while the aggregate allocation is available, enforced as `tokensSold + _amountReceived <= tokenSaleHardCap`.
  - Read their own full purchase history through `getUserPurchases`, and their total credited allocation through `getUserTotalw3o`.
- Disallows:
  - Direct ETH transfers to the contract — `receive` reverts with `"Use`

`buyw3oWithEth instead"`, so ETH may only enter through the signed purchase path. ○ Purchases by any address other than the one named in the signature, since `msg.sender` is bound into the EIP-712 digest. ○ Re-entrant purchases, via `nonReentrant` on both entrypoints. ○ Any on-chain custody, minting, claiming or distribution of the `w3o` token — the contract records allocations only, for export to an external claim portal.	

Code Quality

This audit scope involves the smart contracts of the W3O project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the W3O project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] W3OPresale#buyw3o - Fee-on-transfer and non-standard payment tokens under-fund the treasury while the buyer is credited in full

Description

`buyw3o` accepts an arbitrary `_paymentToken` address and pulls exactly `_amountPaid` from the buyer with `safeTransferFrom`. The contract never measures the treasury balance delta, so it silently assumes that the amount debited from the buyer equals the amount credited to `treasury`. For fee-on-transfer (FOT) / deflationary tokens this assumption is false: the treasury receives `_amountPaid - fee`, yet the buyer is credited the full `_amountReceived` of `w3o`. There is also no on-chain allow-list restricting `_paymentToken`, so any ERC-20 whose transfer semantics deviate from the standard (fee-on-transfer, rebasing, or a token whose balance can shrink) is accepted as long as the off-chain signer signed for it.

Vulnerability Details

The payment leg is a single unchecked pull at `W3OPresale.sol:133`:

```
IERC20(_paymentToken).safeTransferFrom(
    msg.sender,
    treasury,
    _amountPaid
);
```

`SafeERC20.safeTransferFrom` only guarantees that the call did not revert and returned a truthy value — it does **not** guarantee that `treasury` received `_amountPaid`. Accounting is committed *before* the transfer, at `W3OPresale.sol:121-131`:

```
userTotalw3o[msg.sender] += _amountReceived;
tokensSold += _amountReceived;
```

```
userPurchases[msg.sender].push(
    Purchase({
        paymentToken: _paymentToken,
        amountPaid: _amountPaid,
        amountReceived: _amountReceived,
        date: block.timestamp
    })
);
```

So `userTotalw3o`, `tokensSold`, the `Purchase` record and the `w3oPurchased` event all record the *nominal* `_amountPaid`, which is the value the off-chain fulfilment system will reconcile against. The delta between nominal and received value is a direct shortfall for the protocol. The same class of divergence applies to any token with a transfer hook that reduces the delivered amount, and the contract has no `paymentToken` whitelist to constrain which assets can reach this path — the only gate is that `_paymentToken` is part of the signed EIP-712 payload, which pushes the entire validation burden off-chain.

Note also that `_paymentToken` is never checked to be a contract. If the signer signs a payload whose `_paymentToken` is an EOA or an address with no code, `safeTransferFrom` reverts, so that case degrades to a failed purchase rather than a loss.

The intended payment set is USDT, USDC and ETH, none of which charges a transfer fee today — which is why this is Medium rather than High. It is not, however, a safe assumption to bake in. USDT's implementation contains a live fee-on-transfer mechanism (`basisPointsRate` / `maximumFee`, settable by its owner via `setParams`); it is currently zero, but if it is ever enabled the presale silently under-collects on every USDT purchase with no on-chain signal. Since the contract accepts *any* signed `_paymentToken` with no allow-list, the FOT surface is also not actually limited to the three intended assets — it is limited only by backend configuration.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;
```

```
import "forge-std/Test.sol";
import "../src/W3OPresale.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

/// @notice 10% fee-on-transfer token
contract FeeToken is ERC20 {
    constructor() ERC20("Fee", "FEE") {}

    function mint(address to, uint256 amt) external { _mint(to, amt); }

    function _update(address from, address to, uint256 value) internal override {
        if (from != address(0) && to != address(0)) {
            uint256 fee = value / 10;                 // 10% burned on every transfer
            super._update(from, address(0), fee);
            super._update(from, to, value - fee);
        } else {
            super._update(from, to, value);
        }
    }
}

contract FotShortfallTest is Test {
    W3OPresale presale;
    FeeToken   fee;

    uint256 signerPk = 0xA11CE;
    address signer;
    address treasury = address(0xBEEF);
    address buyer    = address(0xCAFE);

    function setUp() public {
        signer  = vm.addr(signerPk);
        presale = new W3OPresale(signer, treasury, 1_000_000e18);
        fee     = new FeeToken();
        fee.mint(buyer, 1_000e18);
        vm.prank(buyer);
        fee.approve(address(presale), type(uint256).max);
    }
```

```
    function test_treasuryReceivesLessThanCredited() public {
        uint256 amountPaid     = 1_000e18;   // buyer pays 1000 FEE
        uint256 amountReceived = 1_000e18;   // credited 1000 w3o (1:1 price)
        uint256 timeSigned     = block.timestamp;
        uint256 salt           = 1;

        bytes memory sig = _sign(buyer, address(fee), amountPaid, amountReceived,
timeSigned, salt);

        vm.prank(buyer);
        presale.buyw3o(address(fee), amountPaid, amountReceived, timeSigned, salt,
sig);

        // Buyer is credited the full nominal amount ...
        assertEq(presale.userTotalw3o(buyer), 1_000e18);
        assertEq(presale.tokensSold(),        1_000e18);

        // ... but the treasury only ever received 90% of it.
        assertEq(fee.balanceOf(treasury), 900e18);

        // 100 FEE of value owed to the protocol simply does not exist.
        assertEq(amountPaid - fee.balanceOf(treasury), 100e18);
    }

    function _sign(
        address _buyer,
        address _token,
        uint256 _paid,
        uint256 _recv,
        uint256 _time,
        uint256 _salt
    ) internal view returns (bytes memory) {
        bytes32 structHash = keccak256(
            abi.encode(
                keccak256(
                    "Buyw3oMessage(address buyer,address paymentToken,uint256
amountPaid,uint256 amountReceived,uint256 timeSigned,uint256 salt)"
```

```
                ),
                _buyer, _token, _paid, _recv, _time, _salt
            )
        );
        bytes32 domain = keccak256(
            abi.encode(
                keccak256("EIP712Domain(string name,string version,uint256
chainId,address verifyingContract)"),
                keccak256("W3OPresale"),
                keccak256("1"),
                block.chainid,
                address(presale)
            )
        );
        bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domain, structHash));
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(signerPk, digest);
        return abi.encodePacked(r, s, v);
    }
}
```

Impact

The protocol receives less value than it credits. Every FOT purchase creates a permanent shortfall equal to the transfer fee, while `userTotalw3o` / `tokensSold` / `w3oPurchased` all record the nominal figure that off-chain fulfilment (the Magna allocation upload) pays out against. With a 10% fee token that is a 10% loss on the entire raise routed through that asset, and the discrepancy is invisible on-chain — reconciliation between the emitted events and the actual treasury balance will simply not add up. Because there is no on-chain payment-token allow-list, whether this triggers depends entirely on which assets the off-chain signer is configured to sign for, so a single mis-configuration, an added payment asset, or USDT enabling its dormant fee switch silently under-funds the sale.

Recommendation

Measure the actual delta received by the treasury and validate it against the signed amount, and restrict payments to an explicit allow-list:

```
+    mapping(address => bool) public allowedPaymentToken;
+
+    event PaymentTokenSet(address indexed token, bool allowed);
+
+    function setPaymentToken(address _token, bool _allowed) external onlyOwner {
+        require(_token != address(0), "Invalid payment token");
+        allowedPaymentToken[_token] = _allowed;
+        emit PaymentTokenSet(_token, _allowed);
+    }
+
     function buyw3o(
         address _paymentToken,
         uint256 _amountPaid,
         ...
     ) external whenNotPaused nonReentrant {
+        require(allowedPaymentToken[_paymentToken], "Payment token not allowed");
+        require(_amountPaid > 0 && _amountReceived > 0, "Zero amount");
         ...
-        IERC20(_paymentToken).safeTransferFrom(
-            msg.sender,
-            treasury,
-            _amountPaid
-        );
+        uint256 balanceBefore = IERC20(_paymentToken).balanceOf(treasury);
+        IERC20(_paymentToken).safeTransferFrom(msg.sender, treasury, _amountPaid);
+        uint256 received = IERC20(_paymentToken).balanceOf(treasury) -
balanceBefore;
+        require(received == _amountPaid, "Fee-on-transfer token not supported");
```

If fee-on-transfer assets must be supported, then credit `_amountReceived` pro-rata to `received / _amountPaid` instead of reverting, and record `received` (not `_amountPaid`) in the `Purchase` struct and the `w3oPurchased` event so off-chain reconciliation uses the real figure.

Status

Resolved

[M-02] W3OPresale#buyw3o - Sale terms are fully off-chain; a compromised adminSigner can drain the entire hard cap for zero payment

Description

Both purchase entrypoints derive *every* economic parameter — the payment token, the amount paid, and the amount of `w3o` credited — from an EIP-712 payload signed by a single `adminSigner` EOA. The contract enforces no relationship between `_amountPaid` and `_amountReceived`: no price, no price floor, no per-user cap, and no minimum payment. The only on-chain invariant is the global `tokenSaleHardCap`. A single compromised or misbehaving signer key therefore mints the entire remaining hard cap of presale credits to arbitrary addresses while paying nothing, and there is no on-chain circuit breaker other than the owner noticing and calling `pause()`.

Vulnerability Details

In `buyw3o` (`W3OPresale.sol:84-146`) and `buyw3oWithEth` (`W3OPresale.sol:148-207`) the full validation set is:

```
require(tokensSold + _amountReceived <= tokenSaleHardCap, "Token sale hard cap reached");
require(!usedSignatures[signatureHash], "Signature already used");
require(_signer == adminSigner, "Invalid signature");
require(block.timestamp <= _timeSigned + 1 hours, "Signature expired");
```

Nothing constrains `_amountReceived` relative to `_amountPaid`. A payload of `(_amountPaid = 0, _amountReceived = tokenSaleHardCap - tokensSold)` passes every check: `tokensSold + _amountReceived <= tokenSaleHardCap` holds, the signature is valid, and `IERC20(_paymentToken).safeTransferFrom(msg.sender, treasury, 0)` succeeds for standard ERC-20s (a zero-value transfer is legal). In the ETH path, `msg.value == _amountPaid == 0` and `treasury.call{value: 0}("")` also succeeds. The purchase is recorded, `userTotalw3o` is credited, and the off-chain fulfilment system sees a legitimate `w3oPurchased` event.

Aggravating factors:

- `adminSigner` is a plain EOA (`W3OPresale.sol:16`), not a multi-sig or threshold signer,

and is set once in the constructor with only a zero-address check.
- Signatures are non-revocable individually. Rotating `adminSigner` via `updateAdminSigner` does invalidate all outstanding signatures from the old key (the check is `_signer == adminSigner`), but that is an all-or-nothing kill switch that also invalidates every legitimate in-flight signature.
- The 1-hour validity window bounds how long a *leaked* signature is usable, but not a *compromised key*, which can mint fresh signatures indefinitely.
- There is no per-address purchase cap, so the drain needs only one transaction from one address.

The mitigations that do exist — `Pausable`, `Ownable2Step`, and `updateAdminSigner` — are all reactive and require the owner to detect the abuse first.

The off-chain pricing model is a deliberate design choice, and a defensible one: the front end quotes live USDT/USDC/ETH rates and applies a tiered bonus (an extra 10% / 15% / 20% of tokens depending on purchase size), which is genuinely awkward to express as a single fixed on-chain price. But "the price is dynamic" only justifies leaving the *exact* rate off-chain — it does not justify leaving the rate *unbounded*. Because the bonus tiers cap the best legitimate deal a buyer can receive at base price plus 20%, a worst-case floor is well defined and can be enforced on-chain without constraining normal quoting.

Proof of Concept

```
function test_compromisedSignerDrainsHardCapForFree() public {
    // Attacker has obtained the adminSigner private key.
    uint256 stolenPk = signerPk;

    uint256 cap        = presale.tokenSaleHardCap();
    uint256 freeTokens = cap - presale.tokensSold();

    address attacker   = address(0xBAD);
    uint256 timeSigned = block.timestamp;

    // amountPaid = 0, amountReceived = the entire remaining hard cap.
```

```
    bytes memory sig = _signWithKey(
        stolenPk, attacker, address(usdc), 0, freeTokens, timeSigned, 1
    );

    vm.prank(attacker);
    presale.buyw3o(address(usdc), 0, freeTokens, timeSigned, 1, sig);

    // Attacker owns the whole presale allocation and paid nothing.
    assertEq(presale.userTotalw3o(attacker), freeTokens);
    assertEq(presale.tokensSold(),           cap);
    assertEq(usdc.balanceOf(treasury),       0);

    // Every legitimate buyer is now permanently locked out.
    bytes memory honestSig = _signWithKey(
        stolenPk, buyer, address(usdc), 1_000e6, 1_000e18, timeSigned, 2
    );
    vm.prank(buyer);
    vm.expectRevert("Token sale hard cap reached");
    presale.buyw3o(address(usdc), 1_000e6, 1_000e18, timeSigned, 2, honestSig);
}
```

Impact

The entire integrity of the sale rests on one hot EOA key. If it leaks — or if the backend that holds it is compromised or has a pricing bug — an attacker mints up to `tokenSaleHardCap - tokensSold` presale credits at zero cost, simultaneously stealing the full token allocation and denying every honest buyer (the hard cap is exhausted, so all subsequent purchases revert). Because the resulting `w3oPurchased` events are structurally indistinguishable from legitimate purchases, the off-chain fulfilment system will honour them. Detection is manual and the only remedy, `pause()`, arrives after the fact. The `w3o` token itself is never held or transferred by this contract, so the loss is realised at distribution time and is not recoverable on-chain.

Recommendation

Add on-chain economic invariants so a signer key alone cannot mint arbitrary value, and keep the signer behind a threshold scheme. Set the floor per payment token so the

differing decimals of USDT/USDC (6) and ETH (18) are handled correctly, and set it loose enough to accommodate the maximum 20% bonus tier — the signer keeps full freedom to quote *above* the floor:

```
+    /// @dev minUnitsPer1e18: minimum payment-token base units per 1e18 w3o,
+    ///      set per token so 6-decimal stablecoins and 18-decimal ETH both work.
+    ///      Configure at the worst legitimate rate: base price less the 20% top
tier.
+    mapping(address => uint256) public minUnitsPer1e18;   // address(0) == ETH
+    uint256 public maxPerUser;
+
+    event PriceFloorSet(address indexed token, uint256 minUnitsPer1e18);
+    event MaxPerUserSet(uint256 maxPerUser);
+
+    function setPriceFloor(address _token, uint256 _minUnits) external onlyOwner {
+        require(_minUnits > 0, "Invalid price floor");
+        minUnitsPer1e18[_token] = _minUnits;
+        emit PriceFloorSet(_token, _minUnits);
+    }
+
+    function _checkTerms(address _token, uint256 _paid, uint256 _received) internal
view {
+        require(_paid > 0 && _received > 0, "Zero amount");
+        uint256 floor = minUnitsPer1e18[_token];
+        require(floor > 0, "Payment token not configured");
+        // Signer may quote any price at or above the floor; it can never
+        // hand out tokens for free or at an arbitrary discount.
+        require(_paid >= (_received * floor) / 1e18, "Price below floor");
+        require(
+            userTotalw3o[msg.sender] + _received <= maxPerUser,
+            "Per-user cap exceeded"
+        );
+    }
+
     function buyw3o(...) external whenNotPaused nonReentrant {
+        _checkTerms(_paymentToken, _amountPaid, _amountReceived);
```

with `_checkTerms(address(0), _amountPaid, _amountReceived)` in `buyw3oWithEth`. Because the floor is owner-settable per token, it can be re-tuned as the ETH price

moves without touching the signer, and the tiered bonus continues to operate entirely off-chain within the allowed band.

Additionally: hold `adminSigner` in an HSM or make it an ERC-1271 multi-sig validated via `SignatureChecker`, support a set of signers with a quorum rather than a single address, emit an event from `updateAdminSigner` so rotation is observable (see [Q-02]), and add a per-window (e.g. hourly) issuance rate limit so that even a valid signer cannot exhaust the cap in a single transaction.

Status

Resolved

Low

[L-01] W3OPresale#buyw3oWithEth - Strict msg.value == _amountPaid equality reverts overpayments instead of refunding the excess

Description

`buyw3oWithEth` requires that `msg.value` exactly equals the signed `_amountPaid`. Any deviation — one wei over, a stale quote, a wallet that rounds, or a signature whose ETH price was computed a block earlier — reverts the whole transaction and burns the buyer's gas. The signed payload already fixes the price, so the safe behaviour is to accept `msg.value >= _amountPaid` and refund the surplus rather than to reject the purchase outright.

Vulnerability Details

At `W3OPresale.sol:160`:

```
require(msg.value == _amountPaid, "ETH amount mismatch");
```

and the forwarding leg at `W3OPresale.sol:197`:

```
(bool success, ) = treasury.call{value: msg.value}("");
require(success, "ETH transfer failed");
```

Because the check is a strict equality, the contract has exactly one acceptable input value, and the caller must reproduce the signed amount to the wei. This is brittle in practice:

- Front-ends and aggregators that pad `msg.value` slightly to absorb price drift will always revert.
- Smart-contract wallets and multi-sig batchers that top up the call value fail for the same reason.
- The buyer cannot recover from a mismatch; each failed attempt costs a full transaction's gas, and with the 1-hour signature window a user who repeatedly mis-sends may run out of validity and need a fresh signature.

There is no residual-ETH risk in the current design (the whole `msg.value` is forwarded), so relaxing to `>=` plus a refund does not leave dust behind — but the refund must be sent *after* the treasury forwarding and must forward `_amountPaid`, not `msg.value`, to the treasury.

Proof of Concept

```
function test_oneWeiOverpaymentBricksThePurchase() public {
    uint256 amountPaid     = 1 ether;
    uint256 amountReceived = 1_000e18;
    uint256 timeSigned     = block.timestamp;

    bytes memory sig = _signEth(buyer, amountPaid, amountReceived, timeSigned, 1);

    vm.deal(buyer, 2 ether);

    // Wallet pads the value by a single wei to absorb price drift.
    vm.prank(buyer);
    vm.expectRevert("ETH amount mismatch");
    presale.buyw3oWithEth{value: amountPaid + 1}(
        amountPaid, amountReceived, timeSigned, 1, sig
    );

    // Same signature, exact value — succeeds. The purchase was only ever
    // one wei away from working, and the user paid gas to learn that.
    vm.prank(buyer);
    presale.buyw3oWithEth{value: amountPaid}(
        amountPaid, amountReceived, timeSigned, 1, sig
    );
    assertEq(presale.userTotalw3o(buyer), amountReceived);
}
```

Impact

Legitimate ETH purchases fail on any non-exact value, wasting buyer gas and creating avoidable drop-off during the presale. Integrators that cannot control `msg.value` to the wei — smart-contract wallets, batchers, front-ends that pad for price movement — are effectively unable to participate. No funds are lost, but conversion is degraded and

repeated failures can outlive the 1-hour signature window, requiring a new signature from the backend.

Recommendation

Accept at least the signed amount, forward exactly the signed amount to the treasury, and refund the remainder to the buyer:

```
-        require(msg.value == _amountPaid, "ETH amount mismatch");
+        require(msg.value >= _amountPaid, "Insufficient ETH");
+        require(_amountPaid > 0 && _amountReceived > 0, "Zero amount");
...
-        (bool success, ) = treasury.call{value: msg.value}("");
-        require(success, "ETH transfer failed");
+        (bool success, ) = treasury.call{value: _amountPaid}("");
+        require(success, "ETH transfer failed");
+
+        uint256 refund = msg.value - _amountPaid;
+        if (refund > 0) {
+            (bool refunded, ) = msg.sender.call{value: refund}("");
+            require(refunded, "Refund failed");
+        }

         emit w3oPurchased(
             msg.sender,
             address(0),
-            msg.value,
+            _amountPaid,
             _amountReceived,
             block.timestamp
         );
```

The refund is placed after the treasury transfer and the function is already `nonReentrant` with all state written beforehand, so the extra external call does not introduce a reentrancy window. Note the event fix as well: `msg.value` must no longer be emitted as the amount paid once overpayment is permitted.

Status

Resolved

[L-02] W3OPresale#updateAdminSigner - Missing zero-address validation permanently bricks the presale

Description

`updateAdminSigner` overwrites the signature-verification authority with no validation at all — the constructor checks `_adminSigner != address(0)` but the setter does not. Setting `adminSigner` to `address(0)` makes every subsequent purchase revert, halting the sale until the owner notices and corrects it.

Vulnerability Details

`W3OPresale.sol:217-219`:

```
function updateAdminSigner(address _newAdminSigner) external onlyOwner {
    adminSigner = _newAdminSigner;
}
```

Contrast with the constructor at `W3OPresale.sol:76`, which does guard the same variable:

```
require(_adminSigner != address(0), "Invalid admin signer address");
```

With `adminSigner == address(0)`, the check at `W3OPresale.sol:114` / `W3OPresale.sol:178` can never pass. `ECDSA.recover` in OpenZeppelin v5 reverts on a malformed signature and never returns `address(0)` for a well-formed one, so there is no path where a forged signature recovers to the zero address — the failure mode is a hard denial of service, not an authentication bypass. Every call to `buyw3o` and `buyw3oWithEth` reverts with `"Invalid signature"` until a valid signer is restored. The setter also emits no event, so the mis-configuration is not visible to off-chain monitoring; the first symptom is users reporting failed purchases.

Proof of Concept

```
function test_zeroAdminSignerHaltsAllPurchases() public {
    uint256 timeSigned = block.timestamp;
```

```
    bytes memory sig = _sign(buyer, address(usdc), 1_000e6, 1_000e18, timeSigned, 1);

    // Owner fat-fingers the rotation and passes the zero address.
    presale.updateAdminSigner(address(0));
    assertEq(presale.adminSigner(), address(0));

    // Every purchase now reverts -- ERC-20 path ...
    vm.prank(buyer);
    vm.expectRevert("Invalid signature");
    presale.buyw3o(address(usdc), 1_000e6, 1_000e18, timeSigned, 1, sig);

    // ... and ETH path.
    bytes memory ethSig = _signEth(buyer, 1 ether, 1_000e18, timeSigned, 2);
    vm.deal(buyer, 1 ether);
    vm.prank(buyer);
    vm.expectRevert("Invalid signature");
    presale.buyw3oWithEth{value: 1 ether}(1 ether, 1_000e18, timeSigned, 2, ethSig);

    // No event was emitted, so nothing off-chain flagged the change.
}
```

Impact

A single mis-typed argument in a privileged transaction takes the presale fully offline: no ERC-20 or ETH purchase can succeed while `adminSigner` is the zero address. The condition is recoverable — the owner simply calls `updateAdminSigner` again — but it is silent (no event) and the sale loses every purchase attempted in the interim, each of which costs the user gas. Requires owner error or a compromised owner, hence low severity.

Recommendation

Validate the input and emit an event so rotations are auditable:

```
+    event AdminSignerUpdated(address indexed oldSigner, address indexed newSigner);
+
     function updateAdminSigner(address _newAdminSigner) external onlyOwner {
+        require(_newAdminSigner != address(0), "Invalid admin signer address");
```

```
+         require(_newAdminSigner != adminSigner, "Signer unchanged");
+         emit AdminSignerUpdated(adminSigner, _newAdminSigner);
          adminSigner = _newAdminSigner;
      }
```

Status

Resolved

[L-03] W3OPresale#updateTreasury - Missing zero-address validation lets ETH proceeds be burned irrecoverably

Description

`updateTreasury` accepts any address, including `address(0)`, while the constructor validates the same variable. With `treasury == address(0)` the two payment paths diverge dangerously: ERC-20 purchases revert (most tokens reject transfers to the zero address), but the ETH path's low-level `call` to `address(0)` **succeeds**, so every ETH purchase burns the buyer's funds while still crediting them `w3o`.

Vulnerability Details

`W3OPresale.sol:221-223`:

```
function updateTreasury(address _newTreasury) external onlyOwner {
    treasury = _newTreasury;
}
```

versus the constructor guard at `W3OPresale.sol:77`:

```
require(_treasury != address(0), "Invalid treasury address");
```

The ETH forwarding at `W3OPresale.sol:197-198` is a raw `call`:

```
(bool success, ) = treasury.call{value: msg.value}("");
require(success, "ETH transfer failed");
```

A `call` to the zero address is a plain value transfer to an account with no code. It does not revert — it returns `success == true` — and the ETH is permanently unrecoverable because no key controls `address(0)`. So the `require(success)` safety net does not fire, the purchase completes, `userTotalw3o` is credited, and `w3oPurchased` is emitted as if the treasury had been funded. The ERC-20 path, by contrast, fails loudly: OpenZeppelin's ERC-20 `_transfer` reverts on a zero recipient, so `safeTransferFrom` bubbles up and the transaction reverts. The result is an inconsistent, silent failure mode that only affects ETH buyers.

Two related gaps in the same setter: `treasury` is not required to be able to receive ETH, so setting it to a contract without a `receive/payable fallback` makes all ETH purchases

revert (a softer DoS), and no event is emitted, so neither mis-configuration is observable off-chain.

Proof of Concept

```
function test_zeroTreasuryBurnsEthProceeds() public {
    presale.updateTreasury(address(0));

    uint256 amountPaid     = 10 ether;
    uint256 amountReceived = 10_000e18;
    uint256 timeSigned     = block.timestamp;

    bytes memory sig = _signEth(buyer, amountPaid, amountReceived, timeSigned, 1);

    vm.deal(buyer, amountPaid);
    uint256 burnedBefore = address(0).balance;

    vm.prank(buyer);
    presale.buyw3oWithEth{value: amountPaid}(
        amountPaid, amountReceived, timeSigned, 1, sig
    );

    // The purchase "succeeded": buyer credited, sale accounting advanced.
    assertEq(presale.userTotalw3o(buyer), amountReceived);
    assertEq(presale.tokensSold(),        amountReceived);

    // But the 10 ETH went to address(0) and is gone forever.
    assertEq(address(presale).balance, 0);
    assertEq(address(0).balance, burnedBefore + amountPaid);

    // The ERC-20 path, by contrast, at least fails loudly.
    bytes memory tokenSig = _sign(buyer, address(usdc), 1_000e6, 1_000e18,
timeSigned, 2);
    vm.prank(buyer);
    vm.expectRevert();
    presale.buyw3o(address(usdc), 1_000e6, 1_000e18, timeSigned, 2, tokenSig);
}
```

Impact

If the owner sets `treasury` to the zero address, all ETH raised while that value is in place is irreversibly burned — the buyers still receive their `w3o` credit and the events look normal, so the loss is only discovered during treasury reconciliation, by which point the ETH is unrecoverable. Setting `treasury` to a contract that cannot accept ETH instead blocks all ETH purchases. Both require a privileged mistake (or compromised owner), and no event is emitted to surface the change, so exposure lasts until someone manually checks the balance.

Recommendation

Validate the address, confirm it can receive ETH, and emit an event:

```
+    event TreasuryUpdated(address indexed oldTreasury, address indexed newTreasury);
+
     function updateTreasury(address _newTreasury) external onlyOwner {
+        require(_newTreasury != address(0), "Invalid treasury address");
+        require(_newTreasury != address(this), "Treasury cannot be self");
+        require(_newTreasury != treasury, "Treasury unchanged");
+        emit TreasuryUpdated(treasury, _newTreasury);
         treasury = _newTreasury;
     }
```

For extra safety, make the treasury rotation two-step (propose / accept) so a contract treasury proves it is reachable before proceeds are routed to it, mirroring the `Ownable2Step` pattern already used for ownership.

Status

Resolved

[L-04] W3OPresale - renounceOwnership is not disabled and would permanently disable pause and all admin functions

Description

The contract inherits `Ownable2Step` without overriding `renounceOwnership`. A single call sets `owner` to `address(0)`, irreversibly disabling `pause`, `unpause`, `updateAdminSigner`, `updateTreasury` and `updateHardCap` — including the emergency stop that is the only defence against the signer-compromise scenario in [M-02].

Vulnerability Details

`Ownable` exposes:

```
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}
```

`Ownable2Step` overrides `transferOwnership` and `acceptOwnership` to add the two-step handshake, but deliberately leaves `renounceOwnership` as a single unguarded call — so the careful two-step protection on transfers does not extend to renunciation. Every privileged function in this contract is `onlyOwner` (`W3OPresale.sol:209-228`), so after renunciation:

- `pause()` is unreachable — the sale cannot be stopped even if `adminSigner` is known to be compromised.
- `updateAdminSigner` is unreachable — a leaked signing key cannot be rotated, and the attacker mints signatures until the hard cap is exhausted.
- `updateTreasury` is unreachable — proceeds are permanently locked to whatever address is set, which if it is a broken or lost address makes the sale unusable or its funds unrecoverable.
- `updateHardCap` is unreachable — the cap is frozen.

There is no recovery: `Ownable` has no mechanism to restore ownership from `address(0)`. The contract is not upgradeable, so redeployment and migration of all off-chain state would be the only path.

Proof of Concept

```
function test_renounceOwnershipBricksEmergencyControls() public {
    // One call, no confirmation step, no timelock.
    presale.renounceOwnership();
    assertEq(presale.owner(), address(0));

    // The emergency stop is gone.
    vm.expectRevert();
    presale.pause();

    // Signer rotation is gone -- a compromised key can no longer be revoked.
    vm.expectRevert();
    presale.updateAdminSigner(address(0xC0FFEE));

    // Treasury and cap are frozen forever.
    vm.expectRevert();
    presale.updateTreasury(address(0xBEEF));
    vm.expectRevert();
    presale.updateHardCap(1);

    // Purchases still work -- the sale runs on with zero admin oversight.
    uint256 timeSigned = block.timestamp;
    bytes memory sig = _sign(buyer, address(usdc), 1_000e6, 1_000e18, timeSigned, 1);
    vm.prank(buyer);
    presale.buyw3o(address(usdc), 1_000e6, 1_000e18, timeSigned, 1, sig);
    assertEq(presale.userTotalw3o(buyer), 1_000e18);
}
```

Impact

Requires an owner mistake or a compromised owner, and the practical effect is loss of all administrative capability — most importantly the `pause()` circuit breaker that [M-02] relies on as its only mitigation. The sale continues accepting purchases with no oversight and no ability to rotate a compromised signer.

Recommendation

Disable renunciation outright, since this contract has no scenario in which ownerless operation is desirable:

```
+    /// @notice Renouncing ownership would permanently disable pause() and
+    ///         adminSigner rotation, so it is intentionally disabled.
+    function renounceOwnership() public view override onlyOwner {
+        revert("Renounce ownership disabled");
+    }
```

Also hold ownership in a multi-sig rather than an EOA, so the emergency stop does not depend on a single key remaining accessible.

Status

Resolved

Gas

[G-01] W3OPresale#buyw3o - Use calldata instead of memory for the _signedMessage parameter

Description

Both external entrypoints declare `bytes memory _signedMessage`, which forces the EVM to copy the 65-byte signature from calldata into memory on every call. Neither function mutates the parameter, so `calldata` is a drop-in replacement that avoids the copy and the associated memory expansion.

Vulnerability Details

`W3OPresale.sol:90` and `W3OPresale.sol:153`:

```
bytes memory _signedMessage
```

`_signedMessage` is used in exactly two places per function — `keccak256(abi.encodePacked(_signedMessage))` and `ECDSA.recover(digest, _signedMessage)` — and OpenZeppelin's `ECDSA.recover(bytes32, bytes memory)` reads the signature without modifying it. Declaring the parameter `calldata` avoids the ABI decoder's copy loop plus the memory allocation for a dynamically-sized `bytes`; the `keccak256` can then hash the calldata slice directly. `ECDSA.recover` requires `bytes memory`, so pass a `calldata`-sliced value or use the `(bytes32 r, bytes32 vs)` overload; the simplest form keeps `recover` fed from a single explicit copy instead of one made on every entry.

Proof of Concept

```
// Baseline (memory) vs optimised (calldata), measured with forge --gas-report:
//
//   buyw3o(bytes memory)   ->  ~ 300 gas of ABI-decode copy + memory expansion
//   buyw3o(bytes calldata) ->  copy elided; signature read in place
//
// Sketch of the optimised form:
function buyw3o(
```

```
    address _paymentToken,
    uint256 _amountPaid,
    uint256 _amountReceived,
    uint256 _timeSigned,
    uint256 _salt,
    bytes calldata _signedMessage      // <-- calldata
) external whenNotPaused nonReentrant {
    bytes32 signatureHash = keccak256(_signedMessage);   // hashes calldata directly
    // ...
    address _signer = ECDSA.recover(digest, _signedMessage);
}
```

Impact

Every purchase — the two hottest functions in the contract — pays an avoidable calldata-to-memory copy plus memory-expansion cost for a 65-byte buffer. The saving is modest per call but applies to every single sale transaction and every buyer, and the change carries no behavioural risk.

Recommendation

```
function buyw3o(
         address _paymentToken,
         uint256 _amountPaid,
         uint256 _amountReceived,
         uint256 _timeSigned,
         uint256 _salt,
-        bytes memory _signedMessage
+        bytes calldata _signedMessage
     ) external whenNotPaused nonReentrant {
```

Apply the identical change to `buyw3oWithEth` at `W3OPresale.sol:153`.

Status

Resolved

[G-02] W3OPresale#buyw3o - Redundant abi.encodePacked when hashing a single bytes value

Description

`keccak256(abi.encodePacked(_signedMessage))` wraps a single dynamic `bytes` argument in `abi.encodePacked`, which for one `bytes` value is the identity operation on the content. It nonetheless allocates a fresh memory buffer and copies all 65 bytes before hashing. `keccak256(_signedMessage)` produces the identical hash with no copy.

Vulnerability Details

`W3OPresale.sol:97` and `W3OPresale.sol:162`:

```
bytes32 signatureHash = keccak256(abi.encodePacked(_signedMessage));
```

`abi.encodePacked` on a single `bytes` argument emits the raw bytes with no length prefix and no padding — exactly the content `keccak256` would hash from the original buffer. The only effect is a memory allocation plus a copy loop, then hashing the copy. Removing the wrapper is bit-for-bit equivalent, so previously issued signature hashes and the `SignatureUsed` event topic remain unchanged; the `usedSignatures` mapping keys are unaffected.

Proof of Concept

```
contract EncodePackedIsRedundant {
    function withWrapper(bytes memory sig) external pure returns (bytes32) {
        return keccak256(abi.encodePacked(sig));   // allocates + copies 65 bytes
    }

    function withoutWrapper(bytes memory sig) external pure returns (bytes32) {
        return keccak256(sig);                     // hashes in place
    }

    function assertEquivalent(bytes memory sig) external pure {
        assert(keccak256(abi.encodePacked(sig)) == keccak256(sig));
    }
}
```

Impact

Each purchase transaction pays for a needless 65-byte memory copy and the memory expansion behind it. Small in isolation, but it sits on the critical path of every sale in both entrypoints, and the fix is provably behaviour-preserving.

Recommendation

```
-       bytes32 signatureHash = keccak256(abi.encodePacked(_signedMessage));
+       bytes32 signatureHash = keccak256(_signedMessage);
```

Apply in both `buyw3o` (`W3OPresale.sol:97`) and `buyw3oWithEth` (`W3OPresale.sol:162`). Combined with [G-01], this hashes the signature straight out of calldata.

Status

Resolved

[G-03] W3OPresale - Replace `require` strings with custom errors

Description

The contract uses fourteen `require` statements with string reason messages. Every such string is stored in the deployed bytecode and ABI-encoded at revert time. Custom errors, available since Solidity 0.8.4 and fully supported by the `0.8.28` pragma in use, encode as a 4-byte selector plus arguments, shrinking deployment size and cutting revert-path gas.

Vulnerability Details

Revert strings appear throughout, e.g. `W3OPresale.sol:92-95`, `W3OPresale.sol:98`, `W3OPresale.sol:114`, `W3OPresale.sol:116`, `W3OPresale.sol:155-158`, `W3OPresale.sol:160`, `W3OPresale.sol:198`, `W3OPresale.sol:226`, `W3OPresale.sol:241`, plus the three constructor guards:

```
require(tokensSold + _amountReceived <= tokenSaleHardCap, "Token sale hard cap
reached");
require(!usedSignatures[signatureHash], "Signature already used");
require(_signer == adminSigner, "Invalid signature");
require(block.timestamp <= _timeSigned + 1 hours, "Signature expired");
require(msg.value == _amountPaid, "ETH amount mismatch");
```

Each distinct string occupies bytecode proportional to its length and, on revert, is ABI-encoded through `Error(string)` — memory allocation plus padding to 32-byte words. `"Token sale hard cap reached"` and `"New cap below tokens already sold"` are long enough to cross into a second word. Custom errors also let the revert carry the offending values (actual vs. expected), which is strictly more useful for integrators than a fixed string.

Proof of Concept

```
error HardCapExceeded(uint256 requested, uint256 remaining);
error SignatureAlreadyUsed(bytes32 signatureHash);
error InvalidSignature(address recovered, address expected);
error SignatureExpired(uint256 timeSigned, uint256 nowTs);
error EthAmountMismatch(uint256 sent, uint256 expected);
```

```
// Before: ~50+ bytes of bytecode per string, Error(string) encoding on revert.
// After:  4-byte selector + packed args.
function buyw3o(...) external whenNotPaused nonReentrant {
    if (tokensSold + _amountReceived > tokenSaleHardCap) {
        revert HardCapExceeded(_amountReceived, tokenSaleHardCap - tokensSold);
    }
    if (usedSignatures[signatureHash]) revert SignatureAlreadyUsed(signatureHash);
    // ...
    if (_signer != adminSigner) revert InvalidSignature(_signer, adminSigner);
    if (block.timestamp > _timeSigned + 1 hours) {
        revert SignatureExpired(_timeSigned, block.timestamp);
    }
}
```

Impact

Meaningful one-time deployment-cost reduction (all fourteen strings leave the bytecode) and lower gas on every reverting call — which matters here because reverts are routine: expired signatures, replayed signatures and hard-cap rejections are all expected outcomes during an active presale. Callers additionally get machine-readable, parameterised revert data.

Recommendation

Declare custom errors at contract scope and convert every `require` to an `if (...) revert Error(...)`. Include the relevant values as arguments — particularly `HardCapExceeded(requested, remaining)`, which lets a front-end display the exact remaining allocation instead of retrying blindly.

Status

Resolved

[G-04] W3OPresale#buyw3o - Pack the Purchase struct to save one storage slot per purchase

Description

`Purchase` occupies four full storage slots: `address paymentToken` (20 bytes, alone in slot 0), then three `uint256` fields. `date` stores a Unix timestamp that cannot exceed `uint48` for any realistic horizon, so narrowing it to `uint48` lets it share slot 0 with `paymentToken`, cutting the struct to three slots and saving a cold `SSTORE` on every purchase.

Vulnerability Details

`W3OPresale.sol:22-27`:

```
struct Purchase {
    address paymentToken;   // slot 0: 20 bytes used, 12 wasted
    uint256 amountPaid;     // slot 1
    uint256 amountReceived; // slot 2
    uint256 date;           // slot 3
}
```

Every purchase pushes one of these into `userPurchases[msg.sender]` at `W3OPresale.sol:124-131`, so each sale performs four zero-to-nonzero `SSTORE`s (20,000 gas each) plus the array-length update. `date` is assigned `block.timestamp`; `uint48` covers timestamps to year 8.9 million, and `uint40` to year 36,812 — either is permanently sufficient. Reordering so `paymentToken` and `date` share slot 0 removes one `SSTORE` entirely:

```
struct Purchase {
    address paymentToken;   // slot 0, bytes 0-19
    uint48  date;           // slot 0, bytes 20-25  -> packed
    uint256 amountPaid;     // slot 1
    uint256 amountReceived; // slot 2
}
```

This is a storage-layout change to a struct held in a mapping-to-array, so it must be applied before deployment (the contract is not upgradeable, so there is no migration concern). The public `userPurchases` getter and `getUserPurchases` return type change `date` to `uint48`, which off-chain consumers must accommodate.

Proof of Concept

```
// Unpacked: 4 cold SSTOREs per purchase   -> ~80,000 gas
// Packed:   3 cold SSTOREs per purchase   -> ~60,000 gas
//           (paymentToken + date co-located in one slot)
//
// ~20,000 gas saved per purchase, on every single sale.

struct PurchasePacked {
    address paymentToken;   // 20 bytes ┐ slot 0
    uint48  date;           //  6 bytes ┘
    uint256 amountPaid;     // slot 1
    uint256 amountReceived; // slot 2
}

userPurchases[msg.sender].push(
    PurchasePacked({
        paymentToken:   _paymentToken,
        date:           uint48(block.timestamp),
        amountPaid:     _amountPaid,
        amountReceived: _amountReceived
    })
);
```

Impact

Roughly one cold `SSTORE` (~20,000 gas) saved per purchase, paid by every buyer on every sale — the single largest per-transaction saving available in this contract. It also shrinks the per-user purchase history footprint by 25%, which reduces the cost of reading it back.

Recommendation

Reorder and narrow the struct as shown, casting at the single write site with `uint48(block.timestamp)`. Keep `amountPaid` and `amountReceived` as full `uint256` since they carry token amounts with 18 decimals. Update any off-chain ABI consumers to expect `uint48` for `date`.

Status

Resolved

[G-05] W3OPresale#buyw3o - Wrap the already-bounds-checked tokensSold increment in unchecked

Description

`tokensSold += _amountReceived` is preceded, in the same function, by `require(tokensSold + _amountReceived <= tokenSaleHardCap)`. That check already proves the sum cannot overflow, yet the compiler emits a second overflow check for the increment. Wrapping it in `unchecked` removes the redundant check.

Vulnerability Details

`W3OPresale.sol:92-95` establishes the bound and `W3OPresale.sol:121-122` performs the writes:

```
require(
    tokensSold + _amountReceived <= tokenSaleHardCap,
    "Token sale hard cap reached"
);
// ...
userTotalw3o[msg.sender] += _amountReceived;
tokensSold += _amountReceived;
```

Under Solidity 0.8.x every arithmetic operation carries an overflow check. Here `tokensSold + _amountReceived` is computed twice — once inside the `require`, once for the assignment — so the contract pays for two `ADD`-plus-branch sequences to enforce a bound that was already proven. Since `tokensSold + _amountReceived <= tokenSaleHardCap <= type(uint256).max`, the increment provably cannot wrap, so `unchecked` is safe.

The same reasoning extends to `userTotalw3o[msg.sender] += _amountReceived`: `userTotalw3o` for any single user is a partial sum of the `_amountReceived` values that also accumulate into `tokensSold`, so it is bounded above by `tokensSold` and therefore by `tokenSaleHardCap`. It cannot overflow either.

A cleaner variant computes the sum once and reuses it, which removes the duplicated `ADD` as well as the duplicated check.

Proof of Concept

```
// Optimised: compute the sum once, reuse it, skip the redundant overflow check.
uint256 newTokensSold = tokensSold + _amountReceived;      // checked once
require(newTokensSold <= tokenSaleHardCap, "Token sale hard cap reached");

// ... signature validation, replay marking ...

unchecked {
    // Safe: newTokensSold <= tokenSaleHardCap, and
    // userTotalw3o[msg.sender] <= tokensSold <= tokenSaleHardCap.
    userTotalw3o[msg.sender] += _amountReceived;
}
tokensSold = newTokensSold;
```

Impact

Removes two redundant overflow checks and one duplicated 256-bit addition from every purchase in both entrypoints. Individually small, but it is pure overhead on the contract's hottest path and the safety argument is airtight given the preceding hard-cap check.

Recommendation

Hoist the sum into a local, `require` against it, assign it directly to `tokensSold`, and wrap the `userTotalw3o` increment in `unchecked` with a comment recording why it cannot overflow. Apply identically in `buyw3oWithEth` (`W3OPresale.sol:155-158`, `W3OPresale.sol:185-186`). Keep the comment — an unexplained `unchecked` block is a maintenance hazard if the hard-cap check is ever moved or removed.

Status

Resolved

QA

[Q-01] W3OPresale - Inconsistent casing and naming conventions across the contract, structs and events

Description

The token is spelled three different ways — `W3O` in the contract name and EIP-712 domain, `w3o` in identifiers, and `W30` in the filename — and the event and struct names violate Solidity naming conventions by starting with a lowercase letter. This is cosmetic but it degrades readability, complicates off-chain integration, and the `W30` vs `W3O` (digit-zero vs letter-O) confusion is an easy source of real mistakes.

Vulnerability Details

The variations, all in one file:

- Filename `W3OPresale.sol` uses `30` (three-zero) while the contract at `W3OPresale.sol:12` is `W3OPresale` (three-letter-O). Visually near-identical, semantically different identifiers.
- Events and structs begin lowercase, against the Solidity style guide's `CapWords` rule: `event w3oPurchased` (`W3OPresale.sol:49`), `struct Buyw3oMessage` (`W3OPresale.sol:29`), `struct Buyw3oWithEthMessage` (`W3OPresale.sol:38`).
- Constants mix cases in a single identifier: `BUY_w3o_MESSAGE_TYPEHASH` (`W3OPresale.sol:61`) and `BUY_w3o_WITH_ETH_MESSAGE_TYPEHASH` (`W3OPresale.sol:66`) embed a lowercase fragment in a `SCREAMING_SNAKE_CASE` name.
- Functions and mappings follow suit: `buyw3o`, `buyw3oWithEth`, `userTotalw3o`, `getUserTotalw3o` — `w3o` is neither capitalised as an acronym (`W30`) nor camel-cased consistently with the surrounding identifiers.

Note that the EIP-712 type strings at `W3OPresale.sol:63` and `W3OPresale.sol:68` must keep matching the struct names exactly — `"Buyw3oMessage(...)"` matches `struct Buyw3oMessage`. Any rename must update the struct, the type string and the off-chain signer together, or every signature will fail to verify.

Proof of Concept

```
// The three spellings, side by side:
//
//   W3OPresale.sol   <- filename: digit zero
//   contract W3OPresale   <- letter O
//   EIP712("W3OPresale", "1")   <- letter O (domain name; must never change
post-deploy)
//   event w3oPurchased(...)     <- lowercase
//   struct Buyw3oMessage        <- lowercase fragment mid-identifier
//   bytes32 constant BUY_w3o_MESSAGE_TYPEHASH   <- lowercase inside
SCREAMING_SNAKE_CASE
//
// A backend author reading the ABI cannot tell from `w3oPurchased` whether the
// on-chain symbol is W3O or W3O, and grepping for the wrong one silently misses.
```

Impact

No security impact; readability and integration-safety only. The `W3O`/`W3O` ambiguity is the practical risk — it invites copy-paste and grep errors when wiring up the off-chain signer and event indexer.

Recommendation

Standardise on one spelling — `W3O` — and apply `CapWords` to types and events, `mixedCase` to functions and variables:

```
-    event w3oPurchased(
+    event W3OPurchased(
-    struct Buyw3oMessage {
+    struct BuyW3OMessage {
-    bytes32 constant BUY_w3o_MESSAGE_TYPEHASH =
+    bytes32 constant BUY_W3O_MESSAGE_TYPEHASH =
-            "Buyw3oMessage(address buyer,...)"
+            "BuyW3OMessage(address buyer,...)"
-    function buyw3o(
+    function buyW3O(
-    mapping(address => uint256) public userTotalw3o;
+    mapping(address => uint256) public userTotalW3O;
```

Rename the file to `W3OPresale.sol` to match the contract. Because the EIP-712 struct names are part of the signed type hash, coordinate the rename with the backend signer in the same release — and leave the `EIP712("W3OPresale", "1")` domain name untouched if the contract is already deployed, since changing it invalidates every signature.

Status

Resolved

[Q-02] W3OPresale#updateHardCap - Privileged setters emit no events

Description

Three of the four owner-only setters mutate security-critical state without emitting an event: `updateAdminSigner` changes who can authorise purchases, `updateTreasury` changes where all proceeds go, and `updateHardCap` changes the supply ceiling. None is observable from logs, so off-chain monitoring cannot alert on them and post-incident forensics must reconstruct the changes by replaying storage.

Vulnerability Details

`W3OPresale.sol:217-228`:

```
function updateAdminSigner(address _newAdminSigner) external onlyOwner {
    adminSigner = _newAdminSigner;
}

function updateTreasury(address _newTreasury) external onlyOwner {
    treasury = _newTreasury;
}

function updateHardCap(uint256 _newHardCap) external onlyOwner {
    require(_newHardCap >= tokensSold, "New cap below tokens already sold");
    tokenSaleHardCap = _newHardCap;
}
```

The contract declares only two events — `w3oPurchased` and `SignatureUsed` — both on the purchase path. Every administrative action is silent. `Pausable` and `Ownable2Step` do emit their own events (`Paused`, `Unpaused`, `OwnershipTransferStarted`, `OwnershipTransferred`), which makes the omission on these three setters inconsistent with the rest of the contract's observability.

This directly compounds [L-02] and [L-03]: a `treasury` or `adminSigner` set to the zero address produces no log line, so the mis-configuration is invisible until purchases start failing or a balance reconciliation comes up short. Raising `tokenSaleHardCap` — an inflation of the presale allocation — is likewise undetectable by watchers.

Proof of Concept

```
function test_adminChangesAreInvisibleInLogs() public {
    vm.recordLogs();

    presale.updateAdminSigner(address(0xDEAD));   // who can authorise sales
    presale.updateTreasury(address(0xBEEF));      // where all money goes
    presale.updateHardCap(type(uint128).max);     // supply ceiling

    Vm.Log[] memory logs = vm.getRecordedLogs();

    // Three critical parameter changes, zero log entries. A monitoring bot
    // subscribed to this address sees nothing at all.
    assertEq(logs.length, 0);

    // Meanwhile the inherited functions DO emit:
    vm.recordLogs();
    presale.pause();
    assertEq(vm.getRecordedLogs().length, 1);     // Paused(address)
}
```

Impact

No direct exploit, but it removes the protocol's ability to monitor its own privileged surface. Combined with the missing zero-address guards, a mis-configuration of `treasury` or `adminSigner` can persist unnoticed while ETH is burned or the sale is offline.

Recommendation

Declare and emit an event from each setter, including the previous value so watchers can diff:

```
+    event AdminSignerUpdated(address indexed oldSigner, address indexed newSigner);
+    event TreasuryUpdated(address indexed oldTreasury, address indexed newTreasury);
+    event HardCapUpdated(uint256 oldHardCap, uint256 newHardCap);
+
     function updateAdminSigner(address _newAdminSigner) external onlyOwner {
+        emit AdminSignerUpdated(adminSigner, _newAdminSigner);
         adminSigner = _newAdminSigner;
```

```
    }

    function updateTreasury(address _newTreasury) external onlyOwner {
+       emit TreasuryUpdated(treasury, _newTreasury);
        treasury = _newTreasury;
    }

    function updateHardCap(uint256 _newHardCap) external onlyOwner {
        require(_newHardCap >= tokensSold, "New cap below tokens already sold");
+       emit HardCapUpdated(tokenSaleHardCap, _newHardCap);
        tokenSaleHardCap = _newHardCap;
    }
```

Status

Resolved

[Q-03] W3OPresale#buyw3o - Missing zero-amount validation permits null purchases that pollute sale accounting

Description

Neither entrypoint requires `_amountPaid > 0` or `_amountReceived > 0`. A signature authorising `(0, 0)` produces a fully successful purchase: a `Purchase` record is appended, `w3oPurchased` is emitted, and a zero-value `safeTransferFrom` executes without reverting for standard ERC-20s. Nothing is exchanged, but the sale's records and event stream are polluted.

Vulnerability Details

The validation set in `buyw3o` (`W3OPresale.sol:92-116`) checks the hard cap, replay, signature validity and expiry — never the amounts. With `_amountPaid == 0` and `_amountReceived == 0`:

- `tokensSold + 0 <= tokenSaleHardCap` passes.
- `IERC20(_paymentToken).safeTransferFrom(msg.sender, treasury, 0)` succeeds — the ERC-20 spec explicitly requires zero-value transfers to be treated as normal transfers.
- In `buyw3oWithEth`, `msg.value == 0 == _amountPaid` passes and `treasury.call{value: 0}("")` returns `true`.
- A `Purchase{amountPaid: 0, amountReceived: 0, date: block.timestamp}` is pushed to `userPurchases[msg.sender]` and `w3oPurchased` fires with zero amounts.

The asymmetric cases are worse than the fully-null one: `_amountReceived > 0` with `_amountPaid == 0` is free tokens (covered in [M-02]), and `_amountPaid > 0` with `_amountReceived == 0` takes the buyer's money and credits nothing. Both are reachable only with a signer-issued signature, which is why this is QA rather than a standalone Medium — but a cheap on-chain guard removes the entire class regardless of backend correctness.

The pollution compounds with the unbounded `userPurchases` array: because the array is only ever appended to and `getUserPurchases` returns the whole thing in one call, an address accumulating many null entries makes its own history progressively more

expensive to read and eventually unreturnable within a block gas limit for on-chain consumers.

Proof of Concept

```
function test_zeroAmountPurchaseSucceeds() public {
    uint256 timeSigned = block.timestamp;

    // Signature authorising a completely null purchase.
    bytes memory sig = _sign(buyer, address(usdc), 0, 0, timeSigned, 1);

    vm.prank(buyer);
    presale.buyw3o(address(usdc), 0, 0, timeSigned, 1, sig);

    // A purchase record now exists for a transaction where nothing happened.
    W3OPresale.Purchase[] memory history = presale.getUserPurchases(buyer);
    assertEq(history.length, 1);
    assertEq(history[0].amountPaid, 0);
    assertEq(history[0].amountReceived, 0);

    // And the worse asymmetric case: buyer pays, receives nothing.
    bytes memory badSig = _sign(buyer, address(usdc), 1_000e6, 0, timeSigned, 2);
    vm.prank(buyer);
    presale.buyw3o(address(usdc), 1_000e6, 0, timeSigned, 2, badSig);

    assertEq(usdc.balanceOf(treasury), 1_000e6);        // money taken
    assertEq(presale.userTotalw3o(buyer), 0);           // nothing credited
}
```

Impact

No value is at risk in the null case, but sale records and the event stream can be filled with meaningless entries, complicating off-chain reconciliation and inflating the per-user history that `getUserPurchases` must return in a single call. The asymmetric variant (payment with no credit) is a direct user loss if the backend ever signs it.

Recommendation

Add an explicit amount guard at the top of both entrypoints, before the expensive signature recovery:

```
) external whenNotPaused nonReentrant {
+        require(_amountPaid > 0, "Zero payment amount");
+        require(_amountReceived > 0, "Zero token amount");
         require(
             tokensSold + _amountReceived <= tokenSaleHardCap,
             "Token sale hard cap reached"
         );
```

Consider also a `minPurchase` threshold, owner-configurable, so dust purchases cannot be used to inflate a user's `userPurchases` array — and bound the read path by adding a paginated `getUserPurchases(address _user, uint256 _offset, uint256 _limit)` alongside the existing full-array getter.

Status

Resolved

[Q-04] W3OPresale#buyw3o - Hard-cap check rejects the entire purchase instead of filling partially

Description

The hard-cap check is all-or-nothing: a purchase that would exceed `tokenSaleHardCap` by any amount reverts in full rather than filling up to the remaining allocation. Near the cap this produces a poor end-of-sale experience — the last tranche of tokens can be effectively unsellable because buyers cannot know the exact remaining amount at execution time.

Vulnerability Details

`W3OPresale.sol:92-95`:

```
require(
    tokensSold + _amountReceived <= tokenSaleHardCap,
    "Token sale hard cap reached"
);
```

Because `_amountReceived` is fixed inside the signed payload, the buyer cannot adjust it to fit the remaining room — a new signature must be requested from the backend for the exact residual amount. This creates a race at the end of the sale: `tokensSold` moves with every confirmed purchase, so a signature quoted against the allocation observed at signing time can be invalidated by any transaction that lands first. Buyers whose transactions revert lose gas and must round-trip to the backend for a fresh quote, and with the 1-hour signature window a sufficiently contested tail can require several attempts.

The revert string is also slightly misleading: it reads "Token sale hard cap reached" when the accurate condition is "this purchase would exceed the remaining allocation" — the cap may not be reached at all, and a smaller purchase would still succeed.

Proof of Concept

```
function test_lastTrancheIsHardToSell() public {
    // Sale is near its cap: 100 tokens remain.
```

```
    uint256 cap = presale.tokenSaleHardCap();
    _fillSaleTo(cap - 100e18);

    uint256 timeSigned = block.timestamp;

    // Buyer was quoted 150 tokens when 200 were still available.
    bytes memory sig = _sign(buyer, address(usdc), 150e6, 150e18, timeSigned, 1);

    vm.prank(buyer);
    vm.expectRevert("Token sale hard cap reached");
    presale.buyw3o(address(usdc), 150e6, 150e18, timeSigned, 1, sig);

    // 100 tokens are still unsold, and the buyer wanted more than that --
    // yet zero tokens were sold and the buyer paid gas for nothing.
    assertEq(cap - presale.tokensSold(), 100e18);

    // The only path forward is a brand-new signature for exactly 100,
    // which any competing purchase landing first will invalidate again.
}
```

Impact

No funds at risk. Buyers near the end of the sale suffer avoidable reverts and wasted gas, and the final tranche of the allocation may go unsold because no buyer can reliably quote it. Purely a UX and sale-completion concern.

Recommendation

Either fill partially up to the remaining allocation and refund the unused payment pro-rata, or expose the remaining amount so front-ends can quote correctly and fail early:

```
function remainingAllocation() external view returns (uint256) {
    return tokenSaleHardCap - tokensSold;
}
```

For partial fills, scale both legs by the same ratio so the price the signer quoted is preserved exactly:

```
uint256 remaining = tokenSaleHardCap - tokensSold;
require(remaining > 0, "Sale fully allocated");

uint256 fillReceived = _amountReceived > remaining ? remaining : _amountReceived;
// Preserve the signed price: charge pro-rata for the filled portion.
uint256 fillPaid = (_amountPaid * fillReceived) / _amountReceived;
```

then transfer only `fillPaid`, credit `fillReceived`, and record the actual filled amounts in the `Purchase` struct and the `w3oPurchased` event. If partial fills are undesirable for accounting reasons, at minimum add `remainingAllocation()` and correct the revert message to reflect the real condition.

Status

Resolved

[Q-05] W3OPresale - Pragma 0.8.28 emits PUSH0, which breaks deployment on chains without EIP-3855

Description

The contract is pinned to `pragma solidity 0.8.28`, which by default targets the Shanghai EVM and emits the `PUSH0` opcode introduced by EIP-3855. Chains and L2s that have not adopted Shanghai treat `PUSH0` as an invalid opcode, so the deployed bytecode reverts. For a presale contract likely to be deployed across several networks, the target EVM version should be pinned explicitly rather than left to the compiler default.

Vulnerability Details

`W3OPresale.sol:2`:

```
pragma solidity 0.8.28;
```

Solidity has defaulted to `evmVersion: shanghai` since 0.8.20 (and `cancun` from 0.8.25 when configured), so `PUSH0` (`0x5f`) appears throughout the compiled output wherever a zero constant is pushed — which in this contract is frequent, given the zero-address comparisons, `address(0)` literals, and empty-`bytes` arguments to `call`. On any chain whose EVM predates Shanghai, `0x5f` is undefined and execution halts with an invalid-opcode error. Historically this has affected a number of production networks and L2s that lag mainnet upgrades.

The failure mode is deployment-time and total rather than subtle — the contract simply does not work on the target chain — so it is a deployment-configuration concern rather than a latent vulnerability. It is worth pinning regardless: the compiler default has changed twice in the 0.8.x series, so relying on it makes the produced bytecode a function of the toolchain version rather than an explicit choice.

The pragma itself is correctly pinned to an exact version (not a floating `^0.8.28`), which is good practice and should be retained.

Proof of Concept

```
# Confirm PUSH0 (0x5f) is present in the compiled output:
forge inspect W3OPresale bytecode | grep -o '5f' | head

# Or inspect the configured EVM version:
solc --version
# solc 0.8.28 defaults to evmVersion: cancun (>=0.8.25) / shanghai (>=0.8.20)

# On a pre-Shanghai chain, deployment or the first zero-push reverts:
#   EvmError: InvalidFEOpcode / invalid opcode: opcode 0x5f not defined
```

```
# foundry.toml -- pin the target explicitly for pre-Shanghai chains
[profile.default]
solc_version = "0.8.28"
evm_version  = "paris"     # no PUSH0 emitted
```

Impact

No security impact on a Shanghai-compatible chain. On a pre-Shanghai chain the contract is simply non-functional — deployment or first execution fails with an invalid opcode — which for a presale would mean a failed launch on that network. Configuration hygiene.

Recommendation

Decide the deployment target set explicitly and pin `evm_version` to match the lowest-common-denominator chain. For mainnet-only or Shanghai-and-later deployments, keep the default and record that decision. For multi-chain deployment, set `evm_version = "paris"` in `foundry.toml` (or `settings.evmVersion` in the Hardhat config) so no `PUSH0` is emitted, and verify the deployed bytecode on each target chain before announcing the sale. Keep the exact-version pragma as-is.

Status

Resolved

Centralisation

The W3O project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the W3O project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with Resolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au